SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 0611 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 10%

QUESTIONS: 2 **Total Marks:20**

Annexure A

MOCK INTERVIEW QUESTIONS

Code of Conduct:

I K.ASWINI(192312397)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1. You are working as a data analyst in a hospital where patient data is continuously collected and stored in CSV files. However, the dataset contains missing values, inconsistent units, and duplicate entries. The system must process this data in near real-time for reporting. Explain how you would design a solution using NumPy and Pandas to clean, filter, and standardize the data efficiently. Discuss indexing, aggregation, and saving the processed data, along with strategies to handle unexpected data anomalies.

15. You are given a dataset with both numerical and categorical features, and you need to prepare it for a machine learning model. How would you perform basic preprocessing such as handling missing values, encoding categorical variables, and normalizing numerical features using **NumPy and Pandas**? Explain the steps briefly.

.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**25**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS

1. Hospital Real-Time Data Cleaning System (NumPy + Pandas Design)

To design an efficient near real-time data processing system for hospital patient CSV data, I would use Pandas as the primary framework for data manipulation and NumPy for fast numerical operations.

First, incoming CSV files would be continuously ingested using a streaming or batch scheduler (like time-based polling). Each dataset would be loaded into a Pandas DataFrame using `read_csv()` with optimized parameters such as `chunksize` for large files.

For data cleaning, missing values would be handled using context-based imputation:

- Numerical fields (e.g., heart rate, BP) → replaced using `mean()`, `median()`, or rolling window statistics for real-time stability.
- Categorical fields (e.g., diagnosis) → filled using `mode()` or a placeholder like `"Unknown"`.
- NumPy's `np.nan` detection would help quickly identify missing entries.

To address inconsistent units, I would standardize values using vectorized Pandas operations:

- Example: converting mg/dL to mmol/L using column-wise transformation.
- NumPy arrays would support fast mathematical scaling across large datasets.

For duplicate removal, `drop_duplicates()` would be applied based on patient ID and timestamp to ensure only the latest or most relevant record is retained.

For indexing and efficiency, I would set a multi-index using `patient_id` and `timestamp`, allowing fast filtering and time-series queries. This improves retrieval speed for reports and analytics.

For aggregation, Pandas `groupby()` would be used to compute metrics like:

- Average vitals per patient
- Daily ward statistics
- Alert thresholds (e.g., abnormal BP counts)

NumPy would support fast aggregation computations when handling large numerical arrays.

To handle unexpected anomalies, such as extreme outliers or corrupted entries:

- Use IQR-based filtering or Z-score detection
- Apply clipping using `np.clip()`
- Validate data ranges (e.g., heart rate must be 30–200)

Finally, the cleaned dataset would be stored using:

- `to_csv()` for reports
- `to_parquet()` for efficient storage and faster retrieval in real-time systems

This design ensures scalability, fast processing, and reliable hospital reporting.

---

15. Data Preprocessing for Machine Learning (NumPy + Pandas Steps)

To prepare a dataset containing both numerical and categorical features for machine learning, I would follow a structured preprocessing pipeline using Pandas for data handling and NumPy for numerical transformations.

First, the dataset is loaded into a Pandas DataFrame using `read_csv()`. Basic inspection using `head()`, `info()`, and `describe()` helps understand missing values and data distribution.

For handling missing values:

- Numerical columns: fill using mean or median (or use `SimpleImputer` logic manually with Pandas)
- Categorical columns: fill using mode or `"Unknown"`
- NumPy helps identify missing values using `np.isnan()` for numerical arrays

For categorical encoding:

- Nominal variables → One-Hot Encoding using `get_dummies()`
- Ordinal variables → Label Encoding by mapping categories to integers
  This ensures machine learning models can interpret categorical data numerically.

For normalizing numerical features:

Min-Max Normalization

$X' = (X - Xmin) / (Xmax - Xmin)$

Z-Score Standardization

$X' = (X - \mu) / \sigma$

Where:

- X = original value
- X' = transformed value
- Xmin = minimum value in dataset
- Xmax = maximum value in dataset
- μ = mean of dataset
- σ = standard deviation of dataset
-

Finally, after preprocessing:

- Combine transformed numerical and encoded categorical features
- Convert into NumPy arrays using `values` or `to_numpy()` for ML model input

This pipeline ensures clean, consistent, and model-ready data for accurate machine learning performance.